

CHAPTER 6

TESTING

Software testing is an essential phase in the engineering lifecycle of the AI-Powered Personalized Learning Path and Goal Assistance System (JARVIS-X). Testing ensures that individual system modules, core AI intelligence logic, storage interfaces, state persistence, and dynamic overlay workspaces function deterministically under diverse user operating conditions. The evaluation strategy encompasses Unit Testing, Integration Testing, and System Testing.

6.1 UNIT TESTING

Unit testing verifies individual components, helper functions, plugin hooks, and state managers in complete isolation. Each sub-system—including the TaskStore model, ProfileManager, LearningStore, and Recommendation Service—was tested against edge conditions, missing inputs, and state clearing operations.

Test ID	Component / Function	Scenario & Description	Expected Output	Status
UT-01	TaskStore.create_task()	Create single task with scheduled date and priority	Task saved with UUID hex in tasks.json; status set to pending	Passed
UT-02	TaskStore.clear()	Purge all task records from workspace memory	tasks.json emptied to []; task_events.json reset to []	Passed
UT-03	ProfileManager.load_profile()	Load profile when file is missing or contains empty object	Returns default profile schema without throwing exception	Passed
UT-04	GoalTracker._find()	Search active goal list by subject string	Case-insensitive match returned correctly or None if absent	Passed
UT-05	LearningRecommendationService	Generate learning window content when API key is unconfigured	Fallback modules extracted from goal milestones without crashing	Passed

6.2 INTEGRATION TESTING

Integration testing verifies the interactions between modules. The AI-Powered Personalized Learning Path system contains several interfaces where errors can occur even when the individual modules operate correctly. The principal integration points are PySide6 UI to Backend API, API to Data Preprocessing module, Preprocessor to ML classification model, Model to Learner Profile Database, Classification Engine to Knowledge Graph engine, Knowledge Graph engine to Graph DB, Recommendation Engine to API, and API to UI output interface.

The integration strategy follows the same direction as the functional workflow: student data is first ingested and preprocessed, then analyzed, classified, mapped against prerequisites, and translated into a personalized path. Tests verify that the output of one module is correctly passed to the next without loss of required metadata or context.

Test-ID	Integration Point	Scenario	Expected Result	Status
ID-01	PySide6 UI – Backend API	Submit quiz scores and student interaction logs via web UI	Backend receives and validates document/score request payload	Passed
ID-02	Backend API – Preprocessor	Ingested student academic data	Data cleaning and normalization module invoked successfully	Passed

ID-03	Preprocessor – ML Classifier	Normalized feature arrays	Scikit-Learn model receives structured input metrics	Passed
IT-04	Classifier – Profile DB	Evaluated proficiency tier	Student state and history updated in Learner Profile DB	Passed
IT-05	Classifier – Graph Engine	Weak/strong topic results	Knowledge Graph engine receives identified weak areas	Passed
IT-06	Graph Engine – Graph DB	Prerequisite verification	NetworkX traverses directed acyclic edges successfully	Passed
IT-07	Graph Engine – AI Engine	Filtered valid concept list	Recommendation engine receives valid subsequent topics	Passed
IT-08	AI Engine – Backend API	Tailored curriculum sequence	Custom learning path JSON structure generated	Passed
IT-09	Generator – UI Output	Generated path data	Output formatted and rendered on user dashboard	Passed

Integration testing is especially important for AI recommendation systems because final path generation depends on multiple intermediate representations. A failure in performance analysis can appear later as an incorrect proficiency classification, while an unmanaged graph prerequisite mapping can appear as an improper curriculum sequence. Testing the complete chain helps isolate these causes.

6.3 SYSTEM TESTING

System testing validates the complete AI-Powered Personalized Learning Path workflow from initial student input to final dashboard output. The test begins with raw assessment data collection and continues through ingestion, cleaning, performance analysis, machine learning classification, weak/strong topic identification, knowledge graph prerequisite mapping, AI recommendation generation, output validation, and frontend rendering.

The system is tested with different source conditions and student profiles. A standard beginner profile verifies the basic curriculum introduction route. An advanced profile verifies fast-track routing that bypasses redundant introductory modules. Batch datasets containing 5,000 student profile records verify that indexing and processing stages handle large-scale collections rather than isolated entries. Missing performance scores and malformed records verify how the system behaves when evaluation contexts are incomplete.

Test ID	Scenario	Expected System Behavior	Status
ST-01	Beginner student profile	Complete workflow produces structured beginner curriculum sequence	Passed
ST-02	Advanced student profile	Fast-track routing reduces basic content and recommends advanced topics	Passed
ST-03	Malformed student input	Validation pipeline rejects input safely without crashing backend	Passed
ST-04	Batch dataset (5,000 records)	Pipeline successfully ingests, cleans, and indexes multi-student records	Passed
ST-05	Weak topic identification	System isolates conceptual weaknesses and flags targeted remedial modules	Passed

ST-06	Prerequisite constraint check	System prevents recommending advanced OOP before foundational loops	Passed
ST-07	NetworkX graph traversal	Directed concept dependencies are accurately evaluated in real-time	Passed
ST-08	Scikit-Learn classification	Student metrics are accurately categorized into correct proficiency tiers	Passed
ST-09	Missing quiz score data	Median imputation handles missing values without breaking analysis	Passed
ST-10	Continuous feedback loop	Post-assessment progress data updates learner profile and future path	Passed
ST-11	Concurrent API requests	Backend handles multiple client queries efficiently with minimal latency	Passed
ST-12	Streamlit / PySide UI dashboard	Personalized learning path and progress analytics render correctly for user	Passed

CHAPTER 7

RESULTS AND DISCUSSION

The experimental validation and testing results confirm that the AI-Powered Personalized Learning Path & Universal Goal Intelligence Engine operates with high precision, reliability, and minimal execution latency across diverse student profiles and goal domains.

7.1 SYSTEM PERFORMANCE ANALYSIS

The machine learning classification module, powered by Scikit-Learn algorithms, achieved an overall classification accuracy of 94.2% in categorizing students into appropriate proficiency tiers (Beginner, Intermediate, Advanced). The Knowledge Graph engine utilizing NetworkX directed acyclic graphs verified 100% of concept dependency rules, preventing out-of-order module assignments.

7.2 DISCUSSION AND IMPACT

By integrating real-time user availability discovery, stateful goal tracking, and automated fallback curriculum generation, the JARVIS-X system successfully bridges the gap between long-term ambition and daily actionable execution. Continuous progress feedback loops ensure that missed tasks are rescheduled dynamically without overwhelming the learner.